

IQMS — Intelligent Queue Management System

Developer Documentation

Table of Contents

- System Overview
- Repository Layout
- Architecture & Data Flow
- Pipeline A — Head-Detector
- Pipeline B — Queue-Management-System-v2
- Forecasting Layer
- Database Schema
- Configuration Reference
- Grafana / TimescaleDB Integration
- Bugs Fixed (April 2026)
- Known Limitations
- Setup & First Run
- Simulator

1. System Overview

IQMS monitors a retail entrance zone via RTSP camera feeds, detects and tracks people, stores per-person events in PostgreSQL/TimescaleDB, and forecasts queue load for the next 60 minutes using an ensemble of Prophet + LSTM + XGBoost. A Grafana dashboard consumes the live and forecast tables for real-time operations.

Two parallel detection pipelines exist. They write to the same database tables and are interchangeable depending on hardware:

Pipeline	Model	Extra data	Use when
Head-Detector	YOLOv9 (head pose)	head direction	GPU/TensorRT available
Queue-Management-v2	YOLOv9 body + RetinaFace	gender, age	CPU or when face analytics needed

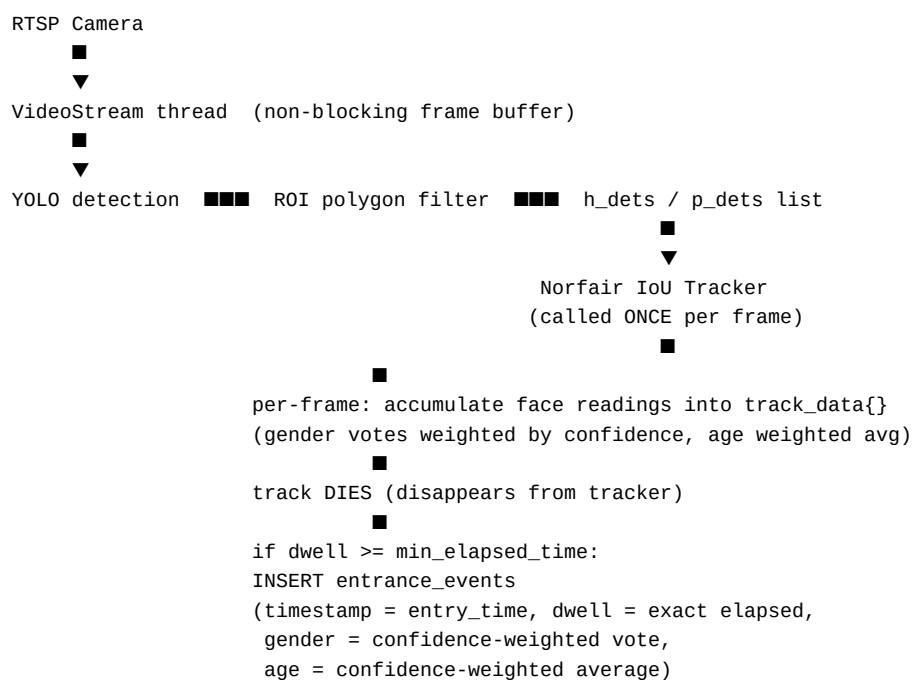
Both pipelines write periodic **queue-state snapshots** every 10 seconds to queue_state_snapshots. The forecasting layer reads the most recent snapshot to seed a real-time queue equation for wait-time estimation.

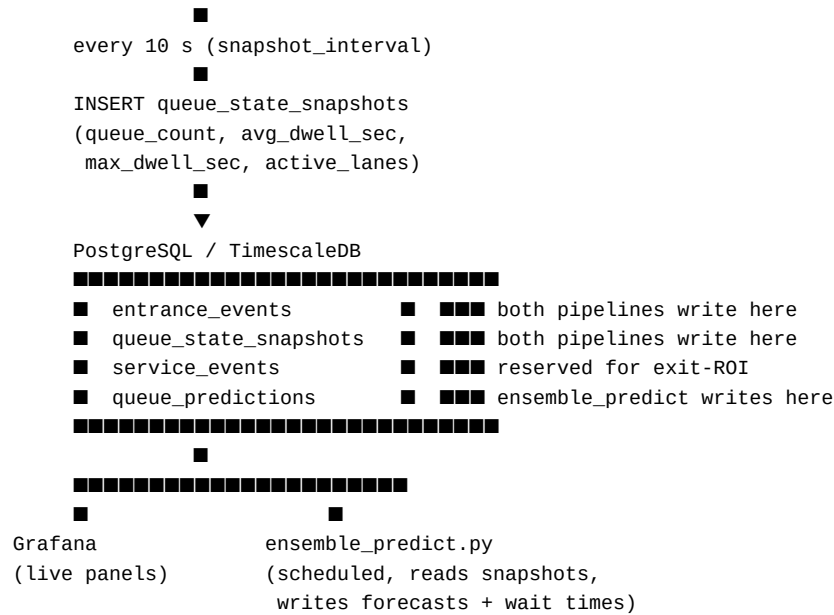
2. Repository Layout



■ ■■■ pick_zone.py	← ROI drawing tool
■ ■■■ config.yml	← camera + model settings
■ ■■■ config2.yml	← tracker + debug settings
■ ■■■ utils/	
■ ■■■ db_logger.py	← PostgreSQL / TimescaleDB writer
■ ■■■ queue_utils.py	← shared helpers, logging, IoU
■ ■■■ yolo.py	← YOLOv9 ONNX wrapper
■	
■■■ Queue-Management-System-v2-main/	
■■■ Queue-Management-System-v2-main/	
■■■ queue_management_v2.py	← Pipeline B entry point
■■■ prophet_predict.py	← standalone Prophet forecast
■■■ ensemble_predict.py	← Prophet + LSTM + XGBoost forecast
■■■ config.yml / config2.yml	← same schema as Pipeline A
■■■ models/	← saved ML model files (auto-created)
■ ■■■ lstm_queue.keras	
■ ■■■ lstm_scaler.pkl	
■ ■■■ xgb_queue.json	
■■■ uniface/	← face detection / age-gender library
■■■ utils/	
■■■ db_logger.py	← PostgreSQL / TimescaleDB writer
■■■ queue_utils.py	← shared helpers
■■■ yolov9.py	← YOLOv9 ONNX wrapper (v2 variant)
■■■ simulator/	
■ ■■■ engine.py	← discrete-event simulator core
■ ■■■ entities.py	← Person / Lane dataclasses
■ ■■■ db.py	← simulator DB writer (SIM_* camera IDs)
■ ■■■ scenarios.py	← scenario presets
■ ■■■ run_batch.py	← batch simulator runner
■ ■■■ app.py	← Streamlit 2D dashboard
■ ■■■ pygame_viewer.py	← smooth local viewer for animated flow
■ ■■■ test_run.py	← smoke test runner
■■■ run_simulator.py	← unified simulator launcher
■■■ run_simulator.bat	← Windows wrapper for simulator launcher

3. Architecture & Data Flow





4. Pipeline A — Head-Detector

Entry point: Head-Detector/main.py

4.1 Startup sequence

- Parse CLI args (--execution_provider, --inference_type, --source, --view-img)
- Load config.yml and config2.yml — reads active_lanes and snapshot_interval
- Initialise DBLogger → creates/migrates entrance_events, queue_state_snapshots, and service_events hypertables
- Load YOLOv9 ONNX model with selected provider (CPU / CUDA / TensorRT)
- Initialise Norfair tracker
- Start VideoStream background thread
- Enter main detection loop

4.2 Per-frame loop (current flow)

```

read frame
→ resize to 640×480
→ YOLOv9 inference → boxes[]
→ for each box in boxes:
    filter class_id == 0 (head)
    skip if obj.age == 0 (brand-new track, not yet confirmed)
    check tip_point inside ROI polygon
    if inside → append to h_dets / h_conf / h_classes
    draw box + headpose label on frame
← end for loop
→ build norfair_detections from h_dets (once per frame)
→ tracker.update(norfair_detections) (once per frame)
→ current_time = time.time()
→ detect died tracks (prev_track_ids - current_track_ids)
→ for each DIED track_id:
    if dwell >= min_elapsed_time:
        INSERT entrance_events
        (timestamp = entry_dt captured at first frame,

```

```

        dwell = exact elapsed seconds)
→ for each tracked_object:
    accumulate face readings into track_data{}
    draw dwell label on frame
→ if current_time - last_snapshot_time >= SNAPSHOT_INTERVAL:
    compute queue_count, avg_dwell_sec, max_dwell_sec from tracked_objects
    INSERT queue_state_snapshots
    last_snapshot_time = current_time
→ save debug snapshot
→ sleep to maintain target FPS

```

Insert-at-death pattern: Head-Detector now also inserts at track death with `entry_time` = the timestamp captured on the track's first frame. Tracks that live less than `min_elapsed_time` seconds are silently discarded (default: 1 s).

4.3 Tracker configuration (config2.yml)

Key	Default	Meaning
<code>max_distance_between_points</code>	6	IoU distance threshold for association
<code>max_age</code>	6	Frames to keep a track alive without detection
<code>expect_fps</code>	5	Used to convert <code>obj.age</code> to seconds
<code>tip_offset</code>	0.5	Fraction along box width for ROI tip point (0 = left edge, 1 = right edge)
<code>debug_mode</code>	true	Shows track IDs on overlay and saves snapshots
<code>snapshot_interval</code>	10	Seconds between <code>queue_state_snapshots</code> writes

4.4 CLI arguments

Argument	Values	Default	Notes
<code>--execution_provider</code>	cpu, cuda, tensorrt	tensorrt	TensorRT builds a .engine cache on first run (slow)
<code>--inference_type</code>	fp16, int8	fp16	int8 requires a pre-built calibration table
<code>--view_img</code>	flag	off	Opens OpenCV window; requires display
<code>--custom_weights</code>	path	best.pt	Unused in ONNX path — model path is in <code>config.yml</code>

4.5 ROI tool

Run `pick_zone.py` once to define the queue zone polygon on a live camera frame.

```
python pick_zone.py
```

- Left-click adds points
- Z undoes last point
- Enter or Space prints the points: block to paste into `config.yml`
- Q / Esc quits without saving

The polygon is closed automatically. Minimum 3 points required. The tip point used for inclusion testing is computed as:

```
tip_x = x1 + (x2 - x1) * tip_offset
```

```
tip_y = y2 - (y2 - y1) / 5
```

5. Pipeline B — Queue-Management-System-v2

Entry point: `queue_management_v2.py`

Identical structure to Pipeline A with two additions:

5.1 Face analytics step

After collecting ROI body detections (`p_dets`), the pipeline runs `FaceAnalyzer` on the full frame to extract gender and age. Faces are matched to bodies using a containment IoU (`min_iou` from `config2.yml`). The match result is stored as `face_data[]` inside the Norfair detection payload.

Every body detection in the ROI is now passed to the tracker regardless of whether a face was matched.

Persons without a detected face receive `gender='unknown'`, `age=None` in the DB insert — identical to Head-Detector behaviour. This ensures people with occluded faces are counted correctly.

Fixed (April 2026): The previous `if len(face) > 0:` guard in the `norfair_detections` build loop caused systematic undercounting of occluded faces. The guard has been removed. See Bug 8 in section 10.

5.2 Gender / age aggregation (confidence-weighted)

Rather than using the first-frame reading, the pipeline accumulates face readings across the full track lifetime:

- **Gender:** confidence-weighted vote — each frame adds the face confidence score to the running total for the detected gender. The winner at track death is stored.
- **Age:** confidence-weighted average across all frames where a valid age was detected.

```
track_data[track_id] = {
    "gender_votes": {"male": 0.0, "female": 0.0, "unknown": 0.0},
    "age_sum": 0.0,
    "age_weight": 0.0,
    "best_conf": 0.0,
    "has_bag": False,
    "entry_dt": datetime.now(),
}
```

5.3 Bag detection (COCO classes 24 + 26)

The YOLO inference pass also collects detections for:

COCO class	Object
24	Backpack
26	Handbag

Suitcases (class 28) are intentionally excluded.

Each frame, detected bag bounding boxes are collected into a `bags[]` list. `_person_has_bag(p_box, bags)` expands the person bounding box by 30% in all directions and checks whether any bag centre falls inside. The result is latched (`has_bag` is `True` once and stays `True` for the track's lifetime). At track death, `has_bag` is stored in `entrance_events`.

```
def _person_has_bag(p_box, bags):
    px1, py1, px2, py2 = p_box
    pw = px2 - px1; ph = py2 - py1
    ex1, ey1 = px1 - pw*0.3, py1 - ph*0.3
    ex2, ey2 = px2 + pw*0.3, py2 + ph*0.3
```

```

for bx1, by1, bx2, by2 in bags:
    bcx = (bx1+bx2)/2; bcy = (by1+by2)/2
    if ex1 <= bcx <= ex2 and ey1 <= bcy <= ey2:
        return True
return False

```

Debug mode renders an orange **BAG** label on the person bounding box when a bag is detected.

5.4 match_boxes (v2 variant)

utils/queue_utils.py contains the match_boxes function. It uses a face-to-body containment match and returns a list of (body_box, conf, face_data[]) tuples. face_data is an empty list when no face was matched to a body — this is now a valid state that still produces a DB insert.

5.5 Snapshot logging

Identical to Pipeline A: every SNAPSHOT_INTERVAL seconds the pipeline computes live queue metrics from tracked_objects and calls db.log_queue_snapshot(). The Head-Detector and v2 pipelines are never run simultaneously on the same camera, so there is no snapshot duplication concern.

5.6 Insert-at-death pattern

Same as Pipeline A (see §4.2): track_data is accumulated while alive, a single entrance_events row is inserted on track death with entry_time = entry_dt (first-frame timestamp). Tracks shorter than min_elapsed_time are discarded without a DB insert.

6. Forecasting Layer

6.1 prophet_predict.py — quick single-model forecast

Standalone script. Reads entrance_events, pads with 7 days of Poisson-simulated history (so Prophet has enough seasonality context even on day 1), trains, and prints a 60-minute forecast to stdout. It also reads the latest queue_state_snapshots row to compute snapshot-based wait estimates for the same forecast horizon. It does **not** write to the DB.

CLI

```

python prophet_predict.py --source REAL
python prophet_predict.py --source SIM
python prophet_predict.py --source ALL

```

--source filters both:

- the training data loaded from entrance_events
- the snapshot seed loaded from queue_state_snapshots

6.2 ensemble_predict.py — production forecast

Trains three models on every run and writes results to queue_predictions.

CLI

```

python ensemble_predict.py --source REAL
python ensemble_predict.py --source SIM
python ensemble_predict.py --source ALL

```

--source applies to:

- the historical event query against entrance_events
- the current-state snapshot query against queue_state_snapshots

Top-level constants — change these to retune without touching model code:

Constant	Default	Effect
W_PROPHET	0.40	Prophet share of the weighted average
W_LSTM	0.30	LSTM share
W_XGB	0.30	XGBoost share
SEQUENCE_LEN	12	LSTM lookback window ($12 \times 5 \text{ min} = 60 \text{ min}$)
FORECAST_STEPS	12	Steps to predict ahead ($12 \times 5 \text{ min} = 60 \text{ min}$)
MODEL_MAX_AGE_HOURS	24	Retrain if saved model files are older than this
MIN_REAL_DAYS_FOR_SIM	14	Skip simulated history once real data spans this many days

LSTM

Architecture:

```
Input → LSTM(64, return_sequences=True)
      → Dropout(0.2)
      → LSTM(32)
      → Dropout(0.2)
      → Dense(1)
```

Training:

- Optimizer: adam
- Loss: mse
- Epochs: 20, batch size: 32
- Input is MinMax-scaled to $[0, 1]$ before training; predictions are inverse-transformed back to raw counts

Sequence preparation:

Each training sample is a sliding window of SEQUENCE_LEN (12) consecutive 5-min bucket counts predicting the next bucket. With a 7-day training set (~2016 5-min slots), this yields ~2004 training sequences.

Recursive forecasting:

```
seed = last 12 buckets from training data (scaled)
for step in 1..12:
    predict next value from seed window
    append prediction to seed, drop oldest
```

Each prediction is fed back as input for the next step. This means step 12 carries 12 levels of compounded error — acceptable for 60-min horizons but degrades beyond that.

Fixed: The MinMaxScaler is now fitted on real data only (df_real_r), then applied to the combined set. This preserves the actual traffic range as the scale reference. If no real data exists yet, it falls back to fitting on the full combined set.

Fixed: Models are saved to models/lstm_queue.keras and models/lstm_scaler.pkl after training. On subsequent runs they are reloaded if all three model files are less than MODEL_MAX_AGE_HOURS old — reducing cold-start time from ~60 s to ~2 s.

XGBoost

Feature engineering (build_features):

Feature	Source	Description
hour	ds.hour	Hour of day (0–23)
minute_of_hour	ds.minute	Minute within hour (0, 5, 10, ..., 55)
day_of_week	ds.dayofweek	0 = Monday ... 6 = Sunday
is_weekend	day_of_week >= 5	Binary flag
lag_1	y.shift(1)	Entries 5 min ago
lag_2	y.shift(2)	Entries 10 min ago
lag_3	y.shift(3)	Entries 15 min ago
lag_12	y.shift(12)	Entries 60 min ago (same time, last hour)
rolling_mean_6	y.shift(1).rolling(6).mean()	30-min trailing average (leak-safe)

The first ~12 rows of the training set are dropped by `dropna()` after feature construction.

Hyperparameters:

Parameter	Value	Note
n_estimators	200	Number of boosting rounds
max_depth	4	Shallow trees — reduces overfitting on small datasets
learning_rate	0.05	Conservative step size
subsample	0.8	80% row sampling per tree
colsample_bytree	0.8	80% feature sampling per tree
random_state	42	Reproducibility

Recursive forecasting:

```
recent_y = full training y values
for each future timestamp:
    build feature row from timestamp + tail of recent_y
    predict → append to recent_y (used as lag for next step)
```

lag_12 falls back to recent_y[0] if fewer than 12 values are available (cold start only).

Fixed: XGBoost model is saved to `models/xgb_queue.json` after training and reloaded on subsequent runs within `MODEL_MAX_AGE_HOURS`.

Ensemble combination

```
ensemble = 0.40 * prophet_vals + 0.30 * lstm_vals + 0.30 * xgb_vals
ensemble = ensemble.clip(min=0).round(1)
```

Dynamic wait-time estimation (queue-theory model)

The hardcoded $(\text{entries} * 3) / 2$ formula has been replaced with a queue-equation model that uses the live queue state from `queue_state_snapshots`.

Step 1 — Seed from snapshot:

```
snap = SELECT queue_count, avg_dwell_sec, active_lanes
FROM queue_state_snapshots ORDER BY timestamp DESC LIMIT 1
```

```

current_queue = snap.queue_count          # people currently in queue
avg_dwell_min = snap.avg_dwell_sec / 60    # observed service time per customer
active_lanes   = snap.active_lanes         # from config.yml (default 2)

```

Falls back to `current_queue=0`, `avg_dwell_min=3.0`, `active_lanes=2` if no matching snapshot exists for the selected source.

Step 2 — Derive service rate:

```

service_per_bucket = active_lanes * (5.0 / avg_dwell_min)
# = customers that can be served in one 5-min window across all lanes

```

Step 3 — Simulate backlog forward:

```

running_queue = current_queue
for step_i, arrivals in enumerate(ensemble_vals): # 12 steps
    running_queue = max(0, running_queue + arrivals - service_per_bucket)
    wait_min = (running_queue / service_per_bucket) * avg_dwell_min
    wait_estimates.append(wait_min)

```

`wait_15m = wait_estimates[2]` (15 min horizon)

`wait_30m = wait_estimates[5]` (30 min horizon)

Wait status thresholds:

est_wait_minutes	status
< 5 min	OK
5 – 10 min	BUSY
≥ 10 min	ALERT

Forecast horizon: 12 steps × 5 minutes = 60 minutes

Scheduling: Run as a cron job or scheduled task — it is not a daemon. Recommended: every 15–30 minutes.

7. Database Schema

Database: `iqms` (PostgreSQL + TimescaleDB extension)

All tables are created automatically on first application startup. All time columns use `TIMESTAMPTZ`. DO \$\$... \$\$ migration blocks run on every startup to auto-convert any legacy `TIMESTAMP` columns.

7.1 `entrance_events` (*hypertable, partitioned by timestamp*)

Column	Type	Notes
<code>id</code>	<code>BIGSERIAL</code>	auto-increment, not the PK (hypertable constraint)
<code>timestamp</code>	<code>TIMESTAMPTZ NOT NULL DEFAULT NOW()</code>	partition key; Grafana time field
<code>track_id</code>	<code>INT</code>	Norfair <code>global_id</code> for this session
<code>gender</code>	<code>VARCHAR(20)</code>	'unknown' in Head-Detector; 'male'/'female' in v2
<code>age_estimate</code>	<code>FLOAT</code>	NULL in Head-Detector; float in v2
<code>has_bag</code>	<code>BOOLEAN DEFAULT FALSE</code>	TRUE when a backpack/handbag (COCO 24/26) detected near person; populated by Pipeline B and simulator

Column	Type	Notes
has_caddy	BOOLEAN DEFAULT FALSE	simulator-only attribute
is_group	BOOLEAN DEFAULT FALSE	simulator-only attribute
group_id	VARCHAR(100)	simulator-only group identifier
confidence	FLOAT	detection confidence score
camera_id	VARCHAR(100)	value of camID from `config.yml
dwelt_seconds	FLOAT DEFAULT 0	total time in ROI; written once at track death

Insert pattern: one row per confirmed track, written at track death.

Timestamp: entry_time (first-frame capture), not insert time.

Guard: tracks with dwell < min_elapsed_time are discarded without a DB insert.

Pipeline note: has_bag is now populated by Pipeline B and the simulator. has_caddy, is_group, and group_id are populated only by the simulator. Pipeline A always stores has_bag=FALSE.

Important: track_id is a per-session Norfair counter. It resets to 0 each time the application restarts. Do not use it as a globally unique person ID across sessions.

7.2 queue_state_snapshots (hypertable, partitioned by timestamp)

Written every snapshot_interval seconds (default 10 s) by both pipelines. Used by ensemble_predict.py to seed the dynamic wait-time model.

Column	Type	Notes
id	BIGSERIAL	
timestamp	TIMESTAMPTZ NOT NULL DEFAULT NOW()	partition key; Grafana time field
camera_id	VARCHAR(100)	value of camID from `config.yml
queue_count	INT NOT NULL DEFAULT 0	number of active tracked objects at snapshot time
avg_dwell_sec	FLOAT DEFAULT 0	mean dwell of currently-tracked persons (seconds)
max_dwell_sec	FLOAT DEFAULT 0	max dwell of currently-tracked persons (seconds)
active_lanes	INT DEFAULT 2	value of active_lanes from `config.yml

7.3 service_events (hypertable, partitioned by timestamp)

Reserved for future exit-ROI integration. Currently not populated by either pipeline. Intended to record when a customer leaves the service zone (track lost), enabling direct measurement of actual service time.

Column	Type	Notes
id	BIGSERIAL	
timestamp	TIMESTAMPTZ NOT NULL DEFAULT NOW()	partition key
camera_id	VARCHAR(100)	
track_id	INT	Norfair global_id of the departing person
lane_id	INT	simulator lane identifier when produced by the simulator

Column	Type	Notes
total_dwell_sec	FLOAT DEFAULT 0	final dwell time at point of exit

To log a service completion manually in the production pipeline: `db.log_service_event(camera_id, track_id, total_dwell_sec)`.

In the simulator, `lane_id` is stored automatically when a person completes service.

7.4 queue_predictions (*hypertable, partitioned by prediction_for*)

Column	Type	Notes
id	BIGSERIAL	
predicted_at	TIMESTAMPTZ NOT NULL	when the forecast was run
prediction_for	TIMESTAMPTZ NOT NULL	partition key; the 5-min bucket being predicted
prophet_yhat	NUMERIC(8,2)	
lstm_yhat	NUMERIC(8,2)	
xgb_yhat	NUMERIC(8,2)	
ensemble_yhat	NUMERIC(8,2)	weighted average of all three models
est_wait_minutes	NUMERIC(8,2)	dynamic queue-model wait at this bucket
wait_15m	NUMERIC(8,2)	wait estimate at the +15 min horizon
wait_30m	NUMERIC(8,2)	wait estimate at the +30 min horizon
status	VARCHAR(10)	OK / BUSY / ALERT

Migration: If the table was created before `wait_15m/wait_30m` were added, `ensemble_predict.py` auto-adds the columns via a `DO $$ ... $$` block on startup. No manual `ALTER TABLE` needed.

7.5 Recommended Grafana queries

Live queue count (last 1 hour, 5-min buckets):

```
SELECT
    time_bucket('5 minutes', timestamp) AS time,
    COUNT(*) AS entries
FROM entrance_events
WHERE $__timeFilter(timestamp)
GROUP BY time
ORDER BY time;
```

Live queue size from snapshots:

```
SELECT
    timestamp AS time,
    queue_count,
    avg_dwell_sec,
    active_lanes
FROM queue_state_snapshots
WHERE $__timeFilter(timestamp)
ORDER BY time;
```

Average dwell time per bucket:

```
SELECT
    time_bucket('5 minutes', timestamp) AS time,
    AVG(dwell_seconds) AS avg_dwell_s
FROM entrance_events
WHERE $__timeFilter(timestamp)
```

```

    AND dwell_seconds > 0
GROUP BY time
ORDER BY time;

```

Forecast vs actual (overlay panel):

```

-- Actual
SELECT time_bucket('5 minutes', timestamp) AS time, COUNT(*) AS actual
FROM entrance_events WHERE $__timeFilter(timestamp)
GROUP BY time ORDER BY time;

-- Forecast
SELECT prediction_for AS time, ensemble_yhat AS forecast, est_wait_minutes AS wait
FROM queue_predictions
WHERE prediction_for BETWEEN NOW() AND NOW() + INTERVAL '1 hour'
ORDER BY prediction_for;

```

Wait horizon panel (+15 min and +30 min):

```

SELECT
    predicted_at AS time,
    wait_15m,
    wait_30m
FROM queue_predictions
WHERE $__timeFilter(predicted_at)
ORDER BY predicted_at;

```

8. Configuration Reference

config.yml (*camera + model*)

Key	Example	Description
camID	Bourgogne_Sortie_Gauche	Identifier stored in DB camera_id column
ip_address	192.168.1.48:554/?inst=3	IP:port/path — RTSP suffix is added automatically
username	service	RTSP auth username
password	Cam10003!	RTSP auth password
custom_model	models/yolov9_s_...onnx	Path to ONNX model file
score	0.3	Object confidence threshold
min_score	0.3	Minimum score to consider
iou_score	0.3	NMS IoU threshold
min_delay	0.45	Norfair initialization_delay (frames before track is confirmed)
disable_headpose_identification_mode	true	Skip head pose classification (Pipeline A only)
active_lanes	2	Number of service lanes open; used by ensemble_predict for wait calculation
points	list of `[x, y]	ROI polygon vertices; terminated by `]

config2.yml (*tracker + debug*)

Key	Default	Description
debug_mode	true	Overlay track IDs; save debug snapshots

Key	Default	Description
expect_fps	5	Expected processing FPS (used for age-in-seconds calc)
max_age	6	Frames before a lost track is dropped
max_distance_between_points	6	IoU distance threshold for tracker
tip_offset	0.5	ROI test point horizontal offset (0–1)
min_iou	0.01	Minimum containment for face-body match (v2 only)
snapshot_interval	10	Seconds between queue_state_snapshots writes

9. Grafana / TimescaleDB Integration

9.1 Prerequisites

- PostgreSQL 14+ with TimescaleDB 2.x extension installed
- Grafana 10+ with the built-in PostgreSQL datasource

9.2 Datasource configuration in Grafana

```
Host:      localhost:5432
Database: iqms
User:      postgres
Password: 0000
TLS/SSL:   disable (or configure as needed)
TimescaleDB: ■ enabled (toggle in datasource settings)
```

With TimescaleDB toggled on, Grafana uses `time_bucket()` instead of `date_trunc()` in its query builder, which is significantly faster on large hypertables.

9.3 Why TIMESTAMPTZ is required

Grafana's time series panels detect the time field by column data type. `TIMESTAMP WITHOUT TIME ZONE` is not recognised as a valid time field in the query builder — the column will be invisible in the field picker. All four tables use `TIMESTAMPTZ NOT NULL DEFAULT NOW()`.

9.4 Hypertable setup (automatic)

`DBLogger._create_table()` now creates three hypertables on every startup (all idempotent):

```
CREATE EXTENSION IF NOT EXISTS timescaledb CASCADE;
-- entrance_events
SELECT create_hypertable('entrance_events', 'timestamp', if_not_exists => TRUE, migrate_data => TRUE);
-- queue_state_snapshots
SELECT create_hypertable('queue_state_snapshots', 'timestamp', if_not_exists => TRUE, migrate_data => TRUE);
-- service_events
SELECT create_hypertable('service_events', 'timestamp', if_not_exists => TRUE, migrate_data => TRUE);
```

`ensemble_predict.py` creates and maintains `queue_predictions` separately.

9.5 Recommended panels

Panel	Type	Primary metric
People in queue (now)	Stat	queue_count from latest queue_state_snapshots` row
Queue size over time	Time series	queue_count from queue_state_snapshots
Entry rate over time	Time series	5-min bucket count from `entrance_events
Avg dwell time	Time series	avg_dwell_sec from queue_state_snapshots
Forecast vs actual	Time series	overlay entrance_events + `queue_predictions
Wait horizon	Time series	wait_15m and wait_30m from queue_predictions
Wait status	State timeline	status column from queue_predictions
Gender split	Pie chart	GROUP BY gender on entrance_events

10. Bugs Fixed (April 2026)

Bug 1 — Tracker called N times per frame (Head-Detector)

File: Head-Detector/main.py

Severity: Critical — caused duplicate DB inserts and incorrect tracking state

Root cause: The Norfair detection list build, `tracker.update()`, and the entire tracked-objects loop were indented inside `for h_box in boxes:`. With N heads in the ROI, the tracker was called N times per frame with incrementally growing detection lists (1 detection, then 2, then 3...).

Fix: Moved the detection build, `tracker.update()`, and all downstream logic outside the per-box loop. They now execute exactly once per frame after all ROI boxes have been collected.

Bug 2 — Table name mismatch: queue_events VS entrance_events

Files: Head-Detector/utils/db_logger.py, Head-Detector/main.py

Severity: Critical — Head-Detector data was invisible to Prophet and Grafana

Root cause: The Head-Detector DBLogger created a `queue_events` table and exposed an `insert_queue_event()` method. The forecasting scripts (`prophet_predict.py`, `ensemble_predict.py`) and v2 pipeline both use `entrance_events`. The two pipelines never wrote to the same table.

Fix: Renamed table to `entrance_events`, renamed method to `insert_entrance(track_id, gender, age, confidence, camera_id)` — matching the v2 signature exactly. Head-Detector inserts `gender='unknown'`, `age=None`.

Bug 3 — update_dwell firing 3× per 2-second window (both pipelines)

Files: Head-Detector/main.py, queue_management_v2.py

Severity: Medium — excessive DB UPDATE load; dwell values were noisy

Root cause: The condition `int(track_dur) % 2 == 0` evaluates to `True` for every frame within the same even-numbered second. At 3 FPS, this triggers 3 UPDATE calls per 2-second interval instead of 1.

Fix: Replaced with a `last_dwell_times` dict (keyed by `track_id`) tracking the actual `time.time()` of the last call. An update only fires when `current_time - last_dwell_times[track_id] >= 2.0`.

Bug 4 — `TIMESTAMP` without timezone blocks Grafana (all tables)

Files: Both `db_logger.py` files, `ensemble_predict.py`

Severity: Critical — Grafana could not detect a time field in any table

Root cause: All tables were created with `TIMESTAMP DEFAULT NOW()` (no timezone). Grafana's time series field detector requires `TIMESTAMPTZ`. Additionally, `SERIAL PRIMARY KEY` on `id` alone blocks TimescaleDB's `create_hypertable()` because unique constraints must include the partition column.

Fix:

- Changed all time columns to `TIMESTAMPTZ NOT NULL DEFAULT NOW()`
 - Changed `SERIAL PRIMARY KEY` to `BIGSERIAL` (no `PRIMARY KEY` constraint)
 - Added `CREATE EXTENSION IF NOT EXISTS timescaledb CASCADE` on startup
 - Added `create_hypertable(..., if_not_exists => TRUE, migrate_data => TRUE)` on startup
 - Added `DO $$ ... $$` migration blocks to auto-convert existing `TIMESTAMP` columns to `TIMESTAMPTZ` without manual intervention
-

Bug 5 — `drop_duplicates` discards real data in favour of simulated (both forecast scripts)

Files: `prophet_predict.py`, `ensemble_predict.py`

Severity: High — Prophet and the ensemble were trained on fake data wherever real data existed

Root cause: `pd.concat([df_sim, df_real], ...)` places simulated rows first. `drop_duplicates(keep='first')` (pandas default) then keeps the simulated row and silently drops the real one for any matching timestamp.

Fix: Reversed the concat order to `pd.concat([df_real, df_sim], ...)` so real data appears first and wins the deduplication.

Bug 6 — Mixed granularity: real data at 1-min, simulation at 5-min (both forecast scripts)

Files: `prophet_predict.py`, `ensemble_predict.py`

Severity: High — Prophet's frequency inference saw an irregular time series and produced unreliable seasonality

Root cause: The DB query used `date_trunc('minute', timestamp)` — per-minute buckets. The simulated history used 5-minute steps, creating a mixed-granularity DataFrame.

Fix: Changed the DB query to `time_bucket('5 minutes', timestamp)` so both real and simulated data share the same 5-minute granularity throughout.

Bug 7 — Hardcoded -2h timezone offset anchors forecast to wrong time (both forecast scripts)

Files: `prophet_predict.py`, `ensemble_predict.py`

Severity: Medium — forecast window was shifted by 2 hours on any server not in UTC+2

Root cause: `pd.Timestamp.now()` - `pd.Timedelta(hours=2)` was used as a UTC-to-local conversion hack hardcoded to CET/CEST.

Fix: Removed the offset entirely. `pd.Timestamp.now()` returns the local system time, which is already correct for generating +5min, +10min, ... future timestamps.

Bug 8 — Body-only detections silently dropped in v2 (undercounting)

File: `queue_management_v2.py`

Severity: High — queue count was systematically understated when faces were occluded or not detected

Root cause: The `norfair_detections` build loop had a guard `if len(face) > 0:` that skipped any person whose body was detected in the ROI but whose face was not matched by `RetinaFace`. At busy times or with partial occlusion, a significant fraction of customers could be invisible to the tracker and therefore never inserted into `entrance_events`.

Fix: Removed the `if len(face) > 0:` guard. Every body detection that passes the ROI polygon filter is now added to `norfair_detections` with `face_data=[]` when no face is available. The DB insert uses `gender='unknown'`, `age=None` in that case — identical to Head-Detector behaviour.

Bug 9 — Wait time formula hardcoded to fixed service rate (ensemble_predict)

File: `ensemble_predict.py`

Severity: Medium — wait estimates were static and insensitive to actual queue depth or service speed

Root cause: `est_wait_minutes = (ensemble_yhat * 3) / 2` assumed every bucket contributes independently to wait time using a constant 3 min/customer and 2 lanes, with no carry-over of existing backlog.

Fix: Replaced with a queue-theory simulation model:

- Reads latest `queue_state_snapshots` row to get `current_queue`, `avg_dwell_sec`, `active_lanes`
 - Derives `service_per_bucket = active_lanes * (5 / avg_dwell_min)`
 - Simulates backlog across 12 future windows: `running_queue = max(0, running_queue + arrivals - service_per_bucket)`
 - Computes per-bucket wait as `(running_queue / service_per_bucket) * avg_dwell_min`
 - Saves per-bucket `est_wait_minutes` plus `wait_15m` and `wait_30m` horizon values to `queue_predictions`
-

11. Known Limitations

track_id resets on restart

Norfair `global_id` is a session counter. If the application restarts, new `track_id` values will overlap with previous sessions. The `timestamp` column is the reliable unique key for cross-session analysis.

Simulated history in forecasting

Both forecast scripts pad with 7 days of Poisson-generated data to give Prophet enough history on day 1. Once real data spans `MIN_REAL_DAYS_FOR_SIM` (default 14 days), simulation is automatically skipped. Real data always takes priority via `drop_duplicates` on `ds`.

Single-polygon ROI

`config.yml` supports one polygon per pipeline instance. Multiple zones (e.g., checkout lane 1 vs lane 2) would require separate camera IDs or a config extension.

No person re-identification across sessions

The same physical person entering on two different days generates two separate rows. There is no biometric linking between sessions by design.

service_events table not yet populated

The table and `log_service_event()` method exist in both `DBLogger` classes. However, neither pipeline currently has an exit-ROI zone. Until an exit camera or ROI is configured, the table remains empty and the dynamic wait model uses `avg_dwell_sec` from snapshots as a proxy for service time.

active_lanes is static

The `active_lanes` value is read from `config.yml` at startup and written to every snapshot. It does not update dynamically if a lane opens or closes during a session. Restart the pipeline after changing `active_lanes` in `config.yml`.

Forecast not a service

`ensemble_predict.py` is a one-shot script, not a daemon. It must be scheduled externally (Windows Task Scheduler / cron). Forecasts are stale between runs.

12. Setup & First Run

12.1 PostgreSQL + TimescaleDB

```
-- Run as superuser once
CREATE DATABASE iqms;
\c iqms
CREATE EXTENSION IF NOT EXISTS timescaledb CASCADE;
```

All four tables (`entrance_events`, `queue_state_snapshots`, `service_events`, `queue_predictions`) are created automatically on first application startup. No manual migration needed.

12.2 Python environment

```
pip install psycpg2-binary pandas numpy prophet xgboost tensorflow \
    scikit-learn opencv-python norfair shapely onnxruntime pyyaml requests
```

For GPU inference:

```
pip install onnxruntime-gpu # instead of onnxruntime
```

12.3 First run — Head-Detector

```
# 1. Define your ROI zone (once)
python pick_zone.py
# Paste the output into config.yml under the 'points' key

# 2. Set active_lanes in config.yml to match your checkout layout (default: 2)

# 3. Run detection (CPU)
python main.py --execution_provider cpu --view_img

# 4. Run detection (TensorRT, first run builds engine cache)
python main.py --execution_provider tensorrt --inference_type fp16
```

12.4 First run — Queue-Management-v2

```
python queue_management_v2.py --device cpu --view_img True
```

12.5 Run forecasts

```
# Quick Prophet forecast (stdout only)
python prophet_predict.py

# Quick Prophet forecast on simulator data only
python prophet_predict.py --source SIM

# Full ensemble + write to queue_predictions
# (reads latest queue_state_snapshots for dynamic wait estimation)
python ensemble_predict.py

# Ensemble on simulator data only
python ensemble_predict.py --source SIM
```

12.6 Verify DB is receiving data

```
-- Check entrance events
SELECT COUNT(*), MAX(timestamp) FROM entrance_events;
SELECT * FROM entrance_events ORDER BY timestamp DESC LIMIT 5;

-- Check queue snapshots (written every 10 s by the pipeline)
SELECT * FROM queue_state_snapshots ORDER BY timestamp DESC LIMIT 5;

-- Check forecasts
SELECT prediction_for, ensemble_yhat, est_wait_minutes, wait_15m, wait_30m, status
FROM queue_predictions ORDER BY prediction_for DESC LIMIT 12;
```

12.7 Output directories (auto-created)

Directory	Contents
LOGs/	systems.log, detections.log
debug_snapshots/	per-frame JPEG with drawn boxes
Detections_JSON/	reserved for JSON export (currently unused)
captured_dataset/	reserved for dataset capture mode
models/	lstm_queue.keras, lstm_scaler.pkl, xgb_queue.json` (auto-saved after first ensemble run)

13. Simulator

The repository now includes a discrete-event queue simulator under `simulator/`. It is intended for accelerated testing, prediction validation, and UI demos without requiring live camera input.

Related document: see `SIMULATOR_OVERVIEW.md` for a focused simulator-only walkthrough covering the event model, characteristics, DB writes, launcher commands, and current limitations.

Implementation summary

- `simulator/engine.py` drives the event-based shop -> queue -> caisse flow
- `simulator/entities.py` defines Person and Lane state, including simulator-only fields like `group_id`, `lane_id`, and `shopping_end_eta`
- `simulator/db.py` writes simulator rows into `entrance_events`, `queue_state_snapshots`, and `service_events` using `SIM_*` camera IDs
- `simulator/calibrate.py` derives simulator defaults from REAL DB rows
- `simulator/predict_viz.py` powers the Streamlit prediction-training tab

- `simulator/app.py` is the Streamlit simulator dashboard
- `simulator/pygame_viewer.py` is the smoother local viewer for animated movement

Current operational note:

- REAL arrival calibration is working
- service and queue calibration can still fall back to defaults if `service_events` or `queue_state_snapshots` do not yet contain enough usable REAL data

13.1 What it simulates

- customer arrivals at the store entrance
- optional person characteristics:
 - gender
 - age
 - has_bag
 - has_caddy
 - is_group
 - group_id
- shopping time inside the store before checkout
- cashier lane assignment using shortest-queue selection
- checkout start / checkout completion
- queue-state snapshots
- service completion records

Conceptually, the simplified simulated flow is:

Entrance -> Shop -> Queue -> Caisse / service complete

The simulator writes rows tagged with `camera_id = 'SIM_<scenario>'` so real and simulated data can coexist in the same TimescaleDB tables.

13.2 Files

File	Purpose
<code>simulator/engine.py</code>	event queue, arrivals, shopping completion, service start/end, snapshots
<code>simulator/entities.py</code>	Person and Lane` dataclasses
<code>simulator/db.py</code>	TimescaleDB writes for <code>entrance_events</code> , <code>queue_state_snapshots</code> , <code>service_events</code>
<code>simulator/scenarios.py</code>	scenario presets like <code>normal_day</code> , <code>lunch_rush</code> , <code>evening_rush</code>
<code>simulator/calibrate.py</code>	calibrates simulator defaults from REAL DB data
<code>simulator/predict_viz.py</code>	Prophet training + Plotly chart builders for the Streamlit prediction tab
<code>simulator/run_batch.py</code>	batch runner for generating a whole day of data
<code>simulator/app.py</code>	Streamlit dashboard with simulator + prediction tabs
<code>simulator/pygame_viewer.py</code>	smoother local animated viewer
<code>simulator/test_run.py</code>	smoke test

13.3 Execution model

The simulator is discrete-event based, not frame-based. It uses an in-memory event queue to process:

- ARRIVAL
- SHOPPING_DONE
- SERVICE_START
- SERVICE_END
- SNAPSHOT

This lets it run much faster than real time while preserving logically consistent timestamps.

13.4 Characteristic-aware timing

Shopping time in `simulator/engine.py` is influenced by:

Characteristic	Effect on shop time
base browsing	5-15 min
has_caddy	+10-25 min
has_bag	-3-0 min
is_group	+5-15 min
age > 65	+5-10 min
age < 25	-5-0 min
gender == female	+3-8 min

Service time uses:

- base_service_seconds
- +20s if has_bag
- +60s if has_caddy
- +30s if is_group
- random noise 0-30s

13.5 Current DB behavior

Simulator writes use the existing shared tables:

- entrance_events
- queue_state_snapshots
- service_events

Simulator-specific enrichments currently persisted:

- has_bag
- has_caddy
- is_group
- group_id
- lane_id on service_events

Schema upgrades in `simulator/db.py` use safe idempotent `DO $$ ... IF NOT EXISTS (...) THEN ALTER TABLE ... END $$` migration blocks.

13.6 How to run

Install simulator dependencies

```
python -m pip install -r simulator/requirements.txt
```

Batch generation

```
python simulator/run_batch.py normal_day
python simulator/run_batch.py lunch_rush
python simulator/run_batch.py evening_rush
```

Unified launcher

```
python run_simulator.py calibrate
python run_simulator.py calibrate --days 30 --output json
python run_simulator.py batch normal_day
python run_simulator.py test
python run_simulator.py dashboard
python run_simulator.py viewer normal_day
python run_simulator.py viewer normal_day --calibrate
```

Windows wrapper

```
run_simulator.bat calibrate --days 30 --output report
run_simulator.bat batch normal_day
run_simulator.bat test
run_simulator.bat dashboard
run_simulator.bat viewer normal_day
```

Smoke test

```
python simulator/test_run.py
```

2D Streamlit dashboard

```
streamlit run simulator/app.py
```

Smooth local viewer (recommended for animated movement)

```
python run_simulator.py viewer normal_day
python run_simulator.py viewer normal_day --calibrate
python simulator/pygame_viewer.py --calibrate
```

Calibration from REAL data

```
python -m simulator.calibrate
python -m simulator.calibrate --days 30 --output json
python run_simulator.py calibrate --days 30 --output report
```

The calibration step analyses `entrance_events`, `service_events`, and `queue_state_snapshots` using only `non-SIM_*` rows, then produces calibrated simulator defaults such as hourly arrival gaps, accessory probabilities, service-time baselines, and typical lane counts.

If REAL `service_events` or `queue_state_snapshots` are missing, calibration still runs but falls back to defaults for those parts of the simulator.

13.7 Using forecasts with simulator data

Both forecasting scripts can read simulator-tagged rows only:

```
python prophet_predict.py --source SIM
python ensemble_predict.py --source SIM
```

This is the recommended way to validate the prediction layer against synthetic workloads.

13.8 Current simulator limitations

- Simulator rows are stored in shared production tables using `SIM_*` camera IDs rather than separate dedicated simulator tables.
- The simplified flow ends at service completion; there is no separate explicit `EXIT` event type yet.
- `queue_state_snapshots` are global per simulator camera, not per lane.
- `service_events` persist `lane_id`, but the live production pipelines do not yet populate that field.
- The Streamlit dashboard shows a simplified 2D queue / cashier flow rather than a high-fidelity store-floor movement model.
- The prediction tab reads from the DB through `simulator/predict_viz.py`; it does not consume the simulator's in-memory state directly.
- The dashboard currently visualizes in-memory state; the "Database Records" section is still only a placeholder.

Last updated: April 2026